

PRACTICAL NO 2

AIM: Implement the A* Search algorithm for solving a pathfinding problem

```
import heapq
```

```
import matplotlib.pyplot as plt
```

```
import networkx as nx
```

```
# -----
```

```
# 1. DATA STRUCTURE POPULATION
```

```
# -----
```

```
# Actual Road Distance g(n) between interconnected nodes (in km)
```

```
graph = {
```

```
    'Mumbai Airport (CSMIA)': {'Sahar Road': 2, 'Vakola / WEH': 4},
```

```
    'Sahar Road': {'Kalina': 5},
```

```
    'Vakola / WEH': {'Kalina': 3, 'Bandra Kurla Complex': 6},
```

```
    'Kalina': {'Kurla West': 4},
```

```
    'Bandra Kurla Complex': {'Kurla West': 4, 'LTT Railway Station': 7},
```

```
    'Kurla West': {'LTT Railway Station': 3},
```

```
    'LTT Railway Station': {}
```

```
}
```

```
# Straight-line Heuristic Distance h(n) from each node to LTT Railway Station
```

```
heuristics = {
```

```
    'Mumbai Airport (CSMIA)': 10,
```

```
    'Sahar Road': 8,
```

```
    'Vakola / WEH': 7,
```

```
    'Kalina': 5,
```

```
    'Bandra Kurla Complex': 6,
```

```
    'Kurla West': 2,
```

```
    'LTT Railway Station': 0
```

```
}
```

```
# Rough spatial coordinates (X, Y) layout mapping from North-West to South-East
```

```
node_positions = {
```

```
    'Mumbai Airport (CSMIA)': (1, 8),
```

```
    'Sahar Road': (2, 7),
```

```
    'Vakola / WEH': (0.5, 6),
```

```
    'Kalina': (2, 5),
```

```
    'Bandra Kurla Complex': (0.5, 4),
```

```
    'Kurla West': (3, 3),
```

```
    'LTT Railway Station': (4, 1)
```

```
}
```

```
# -----
```

```
# 2. A* SEARCH ALGORITHM
```

```
# -----
```

```
def a_star_search(graph, heuristics, start, goal):
```

```
    # Priority Queue stores tuples of: (f_score, current_node, path_taken, cumulative_g_score)
```

```
    priority_queue = [(heuristics[start], start, [start], 0)]
```

```
    visited = set()
```

```
    while priority_queue:
```

```
        # Always extract the node offering the lowest structural  $F(n) = g(n) + h(n)$ 
```

```
        f_score, current, path, g_score = heapq.heappop(priority_queue)
```

```
        if current in visited:
```

```
            continue
```

```
        visited.add(current)
```

```
        # Goal verification
```

```
        if current == goal:
```

```

    return path, g_score

# Explore neighbors
for neighbor, edge_weight in graph[current].items():
    if neighbor not in visited:
        next_g = g_score + edge_weight
        next_f = next_g + heuristics[neighbor]
        heapq.heappush(priority_queue, (next_f, neighbor, path + [neighbor], next_g))

return None, float('inf')

# Run the algorithm
start_node = 'Mumbai Airport (CSMIA)'
goal_node = 'LTT Railway Station'

optimal_path, total_distance = a_star_search(graph, heuristics, start_node, goal_node)
print(f"Optimal Path Discovered: {' -> '.join(optimal_path)}")
print(f"Total Road Distance: {total_distance} km\n")

# -----
# 3. GRAPH PLOTTING AND VISUALIZATION
# -----

# Build the NetworkX graph object
G = nx.DiGraph()

for node, neighbors in graph.items():
    for neighbor, weight in neighbors.items():
        G.add_edge(node, neighbor, weight=weight)

# Initialize plot space
plt.figure(figsize=(12, 9))

```

```
# Define routing edge groupings for distinct coloring

path_edges = list(zip(optimal_path, optimal_path[1:]))

normal_edges = [edge for edge in G.edges() if edge not in path_edges]

# Draw general nodes and layout lines

nx.draw_networkx_nodes(G, node_positions, node_size=3000, node_color='lightblue')

nx.draw_networkx_edges(G, node_positions, edgelist=normal_edges, width=1.5, edge_color='gray',
arrows=True, arrowsize=15)

# Highlight winning traversal path in bold orange

nx.draw_networkx_edges(G, node_positions, edgelist=path_edges, width=4.0,
edge_color='darkorange', arrows=True, arrowsize=20)

# Render structural node text labels including their local h(n) heuristic footprint

node_labels = {node: f"{node}\nh(n)={heuristics[node]}" for node in G.nodes()}

nx.draw_networkx_labels(G, node_positions, labels=node_labels, font_size=9, font_weight='bold')

# Render edge labels displaying actual road distances g(n)

edge_labels = nx.get_edge_attributes(G, 'weight')

formatted_edge_labels = {k: f"{v} km" for k, v in edge_labels.items()}

nx.draw_networkx_edge_labels(G, node_positions, edge_labels=formatted_edge_labels,
font_color='red', font_weight='bold')

plt.title(f"A* Routing Map: {start_node} to {goal_node}\n(Highlighted Path via A*)", fontsize=14,
fontweight='bold')

plt.axis('off')

plt.tight_layout()

plt.show()
```

OUTPUT:

